



UNIVERSITATEA DE VEST DIN TIMIȘOARA
FACULTATEA DE INFORMATICĂ
PROGRAMUL DE STUDII MASTER: Inginerie Software

LUCRARE DE DIZERTAȚIE

MediConnect — Asistent AI pentru Sănătate

*Platformă de Telemedicină cu Plăți Integrate,
Consultatii Video și Inteligență Artificială*

COORDONATOR:
Prof. Dr. Daniel Pop

ABSOLVENT:
Bosna Marinel

TIMIȘOARA
2026



UNIVERSITATEA DE VEST DIN TIMIȘOARA
FACULTATEA DE INFORMATICĂ
PROGRAMUL DE STUDII MASTER: Inginerie Software

LUCRARE DE DIZERTAȚIE

MediConnect — Asistent AI pentru Sănătate

*Platformă de Telemedicină cu Plăți Integrate,
Consultații Video și Inteligență Artificială*

COORDONATOR:
Prof. Dr. Daniel Pop

ABSOLVENT:
Bosna Marinel

TIMIȘOARA
2026

Abstract

MediConnect — AI Healthcare Assistant is a comprehensive telemedicine platform designed to connect patients with both clinic-affiliated and independent medical practitioners in Romania. The system implements a hybrid marketplace architecture built on modern enterprise-grade technologies, offering secure video consultations, intelligent scheduling, integrated payment processing, and AI-assisted medical documentation.

The backend is implemented using Spring Boot 4.0 and Java 21, following a layered Domain-Driven Design architecture with dedicated modules for consultation management, payment processing, video session orchestration, security, and audit logging. The frontend leverages React 19, TypeScript, Vite, and the shadcn/ui component library, offering a fully responsive, accessible, and internationalised user interface supporting both Romanian and English languages.

A key distinguishing feature of the platform is its Stripe Connect Express integration, which implements an escrow-style payment model: patient funds are captured at booking time and transferred to the medical practitioner only after successful consultation completion. The video consultation engine integrates Daily.co WebRTC infrastructure behind a swappable **VideoProvider** interface, ensuring GDPR-compliant EU data residency and HIPAA-grade encryption. Authentication and authorisation are handled through a custom Spring Security implementation with HS256 JSON Web Tokens and role-based access control enforced via CASL on the frontend.

The platform addresses the growing demand for digital health services in post-pandemic Romania, offering a secure, scalable, and legally compliant solution that bridges the gap between patients and healthcare providers across geographical barriers.

Keywords: telemedicine, healthcare platform, Spring Boot, React, Stripe Connect, WebRTC, GDPR, JWT, PostgreSQL, Daily.co

Declarație de integritate științifică

Declar că această lucrare de disertație reprezintă un raport original și constituie propria mea muncă, realizată sub coordonarea Prof. Dr. Daniel Pop. În această lucrare, metodele de cercetare, precum și procesele de colectare, analiză și interpretare a datelor, sunt prezentate corect, iar toate sursele au fost citate corespunzător acolo unde am utilizat idei și/sau materiale aparținând altor autori.

Am utilizat instrumente de inteligență artificială generativă în procesul de redactare și cercetare. Detaliile se regăsesc în Anexa B.

Bosna Marinel

Timișoara, 2026

Cuprins

Lista figurilor	8
Lista tabelelor	9
1 Introducere	10
1.1 Contextul și Motivația Proiectului	10
1.2 Obiectivele Proiectului	10
1.3 Contribuții	11
1.4 Structura Lucrării	11
2 Stadiul Actual și Analiza Soluțiilor Existente	13
2.1 Telemedicina în Context European și Românesc	13
2.2 Analiza Comparată a Platformelor Existente	13
2.3 Inteligența Artificială în Sănătate	14
2.4 Tehnologii WebRTC și Furnizori de Video	14
2.5 Principii de Arhitectură Software Adoptate	14
3 Tehnologii și Instrumente Utilizate	15
3.1 Stiva Tehnologică — Vedere de Ansamblu	15
3.2 Tehnologii Backend	15
3.2.1 Java 21 și Spring Boot 4.0	15
3.2.2 Spring Security și JWT	15
3.2.3 PostgreSQL și JPA/Hibernate	16
3.2.4 Flyway	16
3.2.5 Quartz Scheduler	16
3.3 Tehnologii Frontend	16
3.3.1 React 19 și TypeScript	16
3.3.2 Vite și Ecosistemul de Build	16
3.3.3 TanStack Query	16
3.3.4 Zustand	16
3.3.5 Tailwind CSS și shadcn/ui	17
3.4 Servicii și API-uri Externe	17
3.4.1 Stripe Connect Express	17
3.4.2 Daily.co WebRTC	17
4 Arhitectura Sistemului	18
4.1 Vedere de Ansamblu	18
4.2 Componentele Principale ale Sistemului	18
4.3 Modulele Backend	19

4.4	Fluxul Principal al unei Consultații	19
4.5	Modelul de Date Principal	20
4.6	Mașina de Stări a Consultației	20
5	Implementarea Backend-ului Spring Boot	21
5.1	Structura Proiectului	21
5.2	Entități JPA	21
5.2.1	Entitatea Payment	21
5.2.2	Entitatea Consultation	22
5.3	Serviciile Aplicației	23
5.3.1	ConsultationServiceImpl	23
5.4	Gestionarea Idempotentă a Webhook-urilor	24
5.5	Migrările Flyway	25
5.6	API REST	25
6	Sistemul de Plăți Stripe Connect	27
6.1	Modelul de Plată Escrow	27
6.2	Decizii de Arhitectură Stripe	27
6.3	Chei de Idempotență	28
6.4	Matrice de Scenarii de Eșec	28
6.5	Onboarding Medic pe Stripe Connect	28
6.6	Evenimentele Webhook Stripe Implementate	29
7	Modulul de Consultații Video	30
7.1	Arhitectura VideoProvider	30
7.2	Evenimentele Webhook Video	31
7.3	Automatizarea No-Show	31
7.4	Securitatea Tokenurilor Video	31
7.5	Schema Bazei de Date pentru Video	31
8	Implementarea Frontend-ului React/TypeScript	33
8.1	Arhitectura Frontend	33
8.2	Integrarea Stripe Elements	33
8.3	Componenta StripeStatusBadge	34
8.4	Workspace-ul de Consultație al Medicului	34
8.5	Pagina de Consultație a Pacientului	35
8.6	Controlul Accesului cu CASL	35
8.7	Internaționalizarea Interfeței	36
9	Securitate și Conformitate GDPR	37
9.1	Strategia de Securitate	37
9.2	Criptarea Datelor Medicale	37
9.3	Conformitate GDPR	37
9.4	Conformitate PSD2/SCA	38
9.5	Securitatea Webhook-urilor	38

10 Testarea și Validarea Sistemului	39
10.1 Strategia de Testare	39
10.2 Testele Unitare Backend	39
10.3 Testele de Integrare	40
10.4 Testele Frontend	40
10.5 Carduri de Test Stripe	40
10.6 Criterii de Acceptanță	40
11 Ceea ce a Fost Realizat până în Present	42
11.1 Stadiul Actual al Implementării	42
11.2 Organizarea Ramurilor de Lucru	42
12 Concluzii și Direcții Viitoare de Dezvoltare	44
12.1 Concluzii	44
12.2 Direcții Viitoare de Dezvoltare	45
12.2.1 Faza 2 — Facturare și Taxă (Planificat)	45
12.2.2 Faza 3 — Consultații Instant și Semnalizare Real-Time . . .	45
12.2.3 Faza 4 — Inteligență Artificială Avansată	45
12.2.4 Faza 5 — Scalabilitate și Enterprise	46
12.2.5 Direcții de Cercetare	46
Bibliografie	48
A Variabile de Mediu Necesare	49
B Utilizarea Instrumentelor de Inteligență Artificială Generativă	50
B.1 Detalii privind Utilizarea Claude (Anthropic)	50
B.2 Detalii privind Utilizarea GitHub Copilot	50
C Ghid de Rulare Locală	51
C.1 Pornire Backend	51
C.2 Pornire Frontend	51

Lista figurilor

4.1	Logo Facultatea de Informatică, Universitatea de Vest Timișoara . .	18
-----	---	----

Lista tabelelor

2.1	Analiză comparativă a platformelor de telemedicină	13
4.1	Modulele backend MediConnect	19
5.1	Principalele endpoint-uri API REST	26
6.1	Decizii de arhitectură Stripe	27
6.2	Matrice scenarii de eșec — politici financiare	28
6.3	Evenimente webhook Stripe implementate	29
7.1	Evenimente Daily.co și acțiunile corespunzătoare	31
8.1	Stare — culoare — text per rol (StripeStatusBadge)	34
10.1	Carduri de test Stripe	41
A.1	Variabile de mediu necesare MediConnect	49
B.1	Sinteza utilizării instrumentelor AI generative	50

Capitolul 1

Introducere

1.1 Contextul și Motivația Proiectului

În ultimii ani, digitalizarea sistemului de sănătate a reprezentat una dintre cele mai accelerate transformări din sectorul serviciilor publice și private. Pandemia de COVID-19 a acționat ca un catalizator fără precedent, forțând atât pacienții, cât și medicii să adopte modele noi de interacțiune bazate pe tehnologie [WFC⁺20]. România, țară în care accesul la servicii medicale de specialitate este adesea limitat de bariere geografice, sociale și de infrastructură, a cunoscut o creștere semnificativă a interesului pentru serviciile de telemedicină în această perioadă.

MediConnect este o platformă de telemedicină concepută pentru piața românească, oferind pacienților acces rapid la consultații medicale video cu medici din clinici afiliate sau liberi profesioniști. Platforma abordează trei probleme fundamentale ale sistemului de sănătate digital din România:

1. **Accesibilitatea:** eliminarea barierelor geografice prin consultații video securizate, permițând pacienților din medii rurale sau izolate să beneficieze de consultații de specialitate fără deplasare fizică;
2. **Siguranța financiară:** implementarea unui model de plată de tip escrow (depozit garantat) care protejează atât pacientul, cât și medicul de situații de tip no-show, asigurând corectitudine în tranzacții;
3. **Conformitatea legală:** respectarea cerințelor GDPR (Regulamentul General privind Protecția Datelor), PSD2 (Directiva privind Serviciile de Plată) și a normelor medicale românești privind datele cu caracter special.

Proiectul MediConnect se construiește pe baza cercetărilor recente în domeniile inteligenței artificiale aplicate în medicină [Top19], proceselor de plată online securizate și arhitecturilor software moderne [Ric18].

1.2 Obiectivele Proiectului

Proiectul are următoarele obiective principale:

1. Proiectarea și implementarea unei platforme full-stack de telemedicină care să conecteze pacienții cu medicii într-un ecosistem digital securizat;

2. Integrarea unui sistem de plăți Stripe Connect Express cu model escrow, conform cerințelor PSD2 și cu respectarea TVA-ului românesc;
3. Implementarea unui modul de consultații video bazat pe WebRTC (Daily.co), cu rezidență a datelor în Uniunea Europeană pentru conformitate GDPR;
4. Asigurarea securității datelor prin autentificare JWT, criptare AES-256-GCM a documentelor medicale și control granular al accesului bazat pe roluri (RBAC);
5. Implementarea unui sistem de automatizare a eliberării fondurilor și gestionarea scenariilor de eșec (medic absent, pacient absent, erori tehnice);
6. Oferirea unei interfețe utilizator intuitive, accesibile și internaționalizate (română/engleză).

1.3 Contribuții

Contribuțiile principale ale acestei lucrări sunt:

- **Arhitectură hibrid marketplace:** proiectarea unui sistem multi-rol (pacient, medic independent, medic afiliat clinică, operator) cu politici de acces diferite și fluxuri de lucru dedicate;
- **Model de plată escrow cu Stripe Connect Express:** implementarea completă a fluxului `PaymentIntent` → escrow → transfer automat, cu gestionarea tuturor scenariilor de eșec prin matrice de politici;
- **Integrare video WebRTC transparentă:** abstractizarea furnizorului video prin interfața `VideoProvider`, permițând schimbul furnizorului (Daily.co, Twilio, Agora) fără modificarea logicii de business;
- **Sistem de audit complet:** înregistrarea tuturor tranzacțiilor, tranzițiilor de stare și evenimentelor video într-un jurnal de audit imutabil;
- **Automatizare Quartz JDBC:** job-uri de programare distribuită pentru eliberarea automată a fondurilor și gestionarea scenariilor no-show.

1.4 Structura Lucrării

Lucrarea este structurată în următoarele capitole:

- **Capitolul 2** prezintă stadiul actual al cercetării în domeniile telemedicinii, plăților digitale și WebRTC, precum și analiza soluțiilor existente pe piață;
- **Capitolul 3** descrie tehnologiile și instrumentele utilizate, cu justificarea alegerilor tehnice;
- **Capitolul 4** prezintă arhitectura generală a sistemului, incluzând fluxuri de date și componente principale;
- **Capitolul 5** detaliază implementarea backend-ului Spring Boot;

- **Capitolul 6** acoperă sistemul de plăți Stripe Connect cu modelul escrow;
- **Capitolul 7** prezintă modulul de consultații video cu integrarea Daily.co;
- **Capitolul 8** descrie implementarea frontend-ului React/TypeScript;
- **Capitolul 9** abordează securitatea și conformitatea GDPR;
- **Capitolul 10** prezintă testarea și validarea sistemului;
- **Capitolul 11** conține concluziile și direcțiile viitoare de dezvoltare.

Capitolul 2

Stadiul Actual și Analiza Soluțiilor Existente

2.1 Telemedicina în Context European și Românesc

Telemedicina reprezintă furnizarea de servicii medicale prin intermediul tehnologiilor de telecomunicații, permițând interacțiunea la distanță între pacient și cadrul medical [WFC⁺20]. Conform rapoartelor Organizației Mondiale a Sănătății (OMS), piața globală de telemedicină a depășit valoarea de 80 miliarde USD în 2023 și se estimează că va ajunge la peste 300 miliarde USD până în 2030, cu o rată medie anuală de creștere (CAGR) de peste 18%.

În România, cadrul legal al telemedicinei a fost clarificat prin Ordinul Ministerului Sănătății nr. 1.843/2022, care reglementează consultațiile medicale la distanță. Cu toate acestea, adopția rămâne sub media europeană din cauza infrastructurii digitale limitate în mediul rural, lipsei unui cadru de rambursare prin CNAS, rezistenței culturale din partea unor cadre medicale și absenței unor platforme naționale mature și de încredere.

2.2 Analiza Comparată a Platformelor Existente

Piața de telemedicină prezintă actori diversi, fiecare cu abordări diferite privind gestionarea plăților, furnizarea video și conformitatea GDPR. Tabelul 2.1 prezintă o analiză comparativă a principalelor soluții.

Platformă	Piață	Plăți	Video	GDPR
Teladoc Health	Global	Propriu	WebRTC	Parțial
Doctolib	UE	Propriu	Jitsi	Da
Medicai	România	Card simplu	Zoom	Parțial
RoMedic	România	Fără	Nu	Parțial
MediConnect	România	Escrow/Stripe	Daily.co	Complet

Tabelul 2.1: Analiză comparativă a platformelor de telemedicină

MediConnect se diferențiază prin modelul de plată escrow — o noutate absolută pe piața românească — care protejează ambele părți implicate în consultație, și prin conformarea completă cu GDPR prin rezidența EU a datelor video.

2.3 Inteligența Artificială în Sănătate

Inteligența artificială transformă fundamental modul în care serviciile medicale sunt proiectate și livrate [Top19]. În contextul MediConnect, AI intervine în mai multe puncte ale platformei:

1. **Asistare documentație medicală:** sugestionarea diagnosticărilor și completarea automată a fișelor de consultație pe baza conversației transcrise;
2. **Programare inteligentă:** algoritmi de optimizare a sloturilor disponibile în funcție de istoricul consultațiilor și disponibilitatea medicilor;
3. **Detecție anomalii plăți:** identificarea tranzacțiilor suspecte prin algoritmi de machine learning;
4. **Analiză calitate apel video:** monitorizarea proactivă a calității conexiunii WebRTC și escalarea automată către suport tehnic.

2.4 Tehnologii WebRTC și Furnizori de Video

WebRTC (Web Real-Time Communication) este un standard deschis care permite comunicarea audio/video în timp real direct între browsere, fără plugin-uri suplimentare [SSP15]. Principalele componente ale WebRTC includ: MediaStream API (capturarea audio/video de la microfon și cameră), RTCPeerConnection (stabilirea și menținerea conexiunii peer-to-peer), RTCDataChannel (transfer de date arbitrare între peers) și mecanismele ICE/STUN/TURN pentru traversarea NAT și firewall-urilor.

Analiza furnizorilor WebRTC gestionati a condus la selectarea Daily.co ca furnizor primar, dată fiind superioritatea sa în trei domenii critice: conformitate GDPR cu rezidența EU a datelor (Frankfurt/Dublin), SDK React nativ (@daily-co/daily-react) și profil de criptare HIPAA implicit [Dai24].

2.5 Principii de Arhitectură Software Adoptate

Proiectul adoptă mai multe principii arhitecturale fundamentale [Mar17, Eva03]:

- **Domain-Driven Design (DDD):** organizarea codului în bounded contexts corespunzătoare domeniilor funcționale (booking, consultation, payment, video);
- **Separation of Concerns:** fiecare modul are o responsabilitate bine definită, fără cuplaj strâns;
- **Idempotență:** toate operațiunile externe (webhook-uri, apeluri API) sunt proiectate să poată fi reluate fără efecte secundare;
- **Fail-Fast:** aplicația validează toată configurația necesară la pornire și eșuează cu mesaje clare dacă ceva lipsește.

Capitolul 3

Tehnologii și Instrumente Utilizate

3.1 Stiva Tehnologică — Vedere de Ansamblu

MediConnect adoptă o arhitectură full-stack modernă cu separare clară a responsabilităților. Stiva tehnologică este structurată pe trei straturi principale: stratul de prezentare (frontend React/TypeScript), stratul de logică de business (backend Spring Boot/Java) și stratul de date (PostgreSQL).

3.2 Tehnologii Backend

3.2.1 Java 21 și Spring Boot 4.0

Backend-ul MediConnect este implementat în Java 21 folosind Spring Boot 4.0.5 [Wal16]. Alegerea acestei versiuni de Java aduce avantaje semnificative: Virtual Threads (Project Loom) — fire de execuție ușoare, gestionate de JVM, care reduc semnificativ overhead-ul pentru operații I/O; Record classes — simplificarea definiției obiectelor de transfer de date (DTO); Pattern matching — cod mai concis pentru logica de dispatch a evenimentelor webhook; Sealed classes — modelarea ierarhiilor de stare ale mașinii de stare a plăților.

Spring Boot 4.0 oferă auto-configurare, management integrat al dependențelor prin Maven și un server web embedded (Tomcat), permițând deployment simplu ca JAR executabil.

3.2.2 Spring Security și JWT

Autentificarea și autorizarea sunt gestionate printr-o implementare custom a Spring Security, bazată pe JSON Web Tokens (JWT) cu algoritmul HS256 [JBS15, VMw24]. Tokenurile au o durată de viață de 24 de ore și conțin următoarele claims: identificatorul unic al utilizatorului (UUID), lista de roluri (PATIENT, MEDIC, OPERATOR), timestamp-ul de expirare și cel al emiterii.

3.2.3 PostgreSQL și JPA/Hibernate

Baza de date este PostgreSQL 16, recunoscută pentru robustețe, suport ACID complet și capabilități avansate de tip JSONB pentru stocarea payload-urilor de web-hook [The24]. Accesul la date se face prin JPA (Jakarta Persistence API) cu Hibernate ca implementare ORM, cu blocare optimistă prin câmpul `@Version` pentru prevenirea condițiilor de concurență.

3.2.4 Flyway

Gestionarea schimbărilor de schemă a bazei de date se realizează cu Flyway [Red24], prin fișiere SQL numerotate care asigură reproducibilitatea completă a schemei pe orice mediu (development, testing, production).

3.2.5 Quartz Scheduler

Sarcinile planificate sunt gestionate de Quartz Scheduler în configurație JDBC Job-StoreTX [Ter24], care asigură rezistența la restartări și funcționarea corectă în medii cluster.

3.3 Tehnologii Frontend

3.3.1 React 19 și TypeScript

Interfata utilizator este construită cu React 19 și TypeScript 5.x în mod strict [Mic24, BP20]. TypeScript aduce verificare statică a tipurilor, prevenind o categorie întreagă de erori la runtime, cu impact deosebit în modulele critice de plăți și video.

3.3.2 Vite și Ecosistemul de Build

Toolchain-ul de build este Vite [You24], care oferă HMR (Hot Module Replacement) instantaneu în development, build-uri de producție optimizate prin Rollup cu tree-shaking și suport nativ ESM (ECMAScript Modules).

3.3.3 TanStack Query

Gestionarea stării serverului se face prin TanStack Query [Lin24], care oferă cache inteligent cu invalidare automată, refetch în background, optimistic updates și polling configurabil per query.

3.3.4 Zustand

Starea globală a clientului este gestionată prin Zustand [Kat24], o bibliotecă minimală de state management bazată pe hook-uri React, fără boilerplate.

3.3.5 Tailwind CSS și shadcn/ui

Stilizarea UI se face cu Tailwind CSS v4 [WR24], un framework utility-first. Componentele complexe sunt furnizate de shadcn/ui, o colecție de componente accesibile construită pe Radix UI.

3.4 Servicii și API-uri Externe

3.4.1 Stripe Connect Express

Procesarea plăților se realizează prin Stripe Connect Express [Str24], model ce permite plăți către medici ca entități distincte (subconturi) cu transfer automat al fondurilor din contul platformei. Stripe este liderul global în procesarea plăților online, cu suport nativ pentru PSD2/SCA și conformitate PCI DSS.

3.4.2 Daily.co WebRTC

Consultațiile video sunt alimentate de Daily.co [Dai24], o platformă WebRTC gestionată ce oferă SDK-uri native pentru React și un sistem de webhook-uri pentru sincronizarea stării sesiunii cu backend-ul.

Capitolul 4

Arhitectura Sistemului

4.1 Vedere de Ansamblu

MediConnect implementează o arhitectură în trei straturi, cu separație clară între logica de prezentare, logica de business și accesul la date. Sistemul este structurat ca o aplicație monolitică modulară (modular monolith), o alegere deliberată care combină simplitatea deploy-ului monolitic cu organizarea clară specifică microserviciilor [Mar17, Ric18].



Figura 4.1: Logo Facultatea de Informatică, Universitatea de Vest Timișoara

4.2 Componentele Principale ale Sistemului

Sistemul este compus din următoarele componente majore:

1. **Frontend SPA:** aplicație React/TypeScript care rulează în browser, comunicând cu backend-ul prin API REST și cu Daily.co direct prin SDK WebRTC;
2. **Backend Spring Boot:** serviciu REST cu mai multe module funcționale, expunând endpoint-uri securizate JWT;
3. **PostgreSQL:** baza de date relațională centrală;
4. **Stripe:** procesator de plăți extern, comunicat prin API REST și webhook-uri;
5. **Daily.co:** furnizor WebRTC extern, comunicat prin API REST (creare camere, tokenuri) și webhook-uri (evenimente sesiune);
6. **Quartz JDBC:** sistem de planificare distribuită pentru job-uri automate.

4.3 Modulele Backend

Backend-ul este organizat în module cu responsabilități bine definite, așa cum este prezentat în Tabelul 4.1.

Modul	Responsabilitate
auth	Autentificare, generare/validare JWT, gestionare utilizatori
booking	Gestionarea rezervărilor, sloturi disponibile, calendar
consultation	Ciclul de viață al consultației (SCHEDULED → IN_PROGRESS → COMPLETED)
payment	Escrow Stripe, PaymentIntent, Transfer, Refund, Dispute
video	Creare camere Daily.co, tokenuri de acces, webhook video
medic	Profiluri medici, specializări, clinici afiliate
notification	Evenimente de domeniu, notificări (placeholder Faza 2)
audit	Jurnal imutabil al tuturor acțiunilor semnificative
webhook	Receptor Stripe și Daily webhooks, idempotență

Tabelul 4.1: Modulele backend MediConnect

4.4 Fluxul Principal al unei Consultații

Fluxul complet al unei consultații MediConnect parcurge următorii pași:

1. Pacientul selectează medicul, specialitatea și slotul orar;
2. Pacientul completează plata prin Stripe (PaymentIntent — fonduri capturate, stare RESERVED);
3. Backend-ul confirmă rezervarea, creează entitatea Consultation (stare SCHEDULED);
4. Cu 15 minute înainte de start, backend-ul creează camera Daily.co via API;
5. La ora programată, medicul apasă Start — backend creează tokenuri de acces (OWNER);
6. Pacientul primește token (PARTICIPANT), intră în sala de așteptare pre-call;
7. Ambele părți se alătură camerei video — Daily webhook `meeting.started` trigger-ează starea IN_PROGRESS;
8. Stripe capturează fondurile (escrow avansat la HELD);
9. La finalul consultației, medicul apasă End — Daily webhook `meeting.ended`;
10. Backend completează consultația, calculează durata;
11. După fereastra de dispute (120 min), Quartz job eliberează fondurile către medic (Transfer Stripe).

4.5 Modelul de Date Principal

Entitățile centrale ale sistemului sunt: **User** (utilizator cu roluri multiple), **Booking** (rezervare legată de slot, medic și pacient), **Consultation** (sesiunea medicală efectivă, 1:1 cu Booking), **Payment** (tranzacția financiară Stripe, cu mașină de stare), **MedicStripeAccount** (contul Stripe Connect al medicului) și **WebhookEvent** (eveniment webhook primit, cu idempotență).

4.6 Mașina de Stări a Consultației

Consultația parcurge un ciclu de viață bine definit, implementat prin enumul `ConsultationStatus`:

SCHEDULED → IN_PROGRESS → COMPLETED

SCHEDULED → FAILED

IN_PROGRESS → FAILED

IN_PROGRESS → DISPUTED

Fiecare tranziție este validată explicit în stratul de servicii, prevenind tranziții invalide (de ex., trecerea direct din SCHEDULED în COMPLETED).

Capitolul 5

Implementarea Backend-ului Spring Boot

5.1 Structura Proiectului

Proiectul backend este un monolit modular organizat în pachete Java corespunzătoare domeniilor funcționale:

Listing 5.1: Structura proiectului backend

```
1 src/main/java/org/example/healthcare/  
2   auth/          -- autentificare si JWT  
3   booking/       -- rezervari si sloturi  
4   consultation/  -- ciclul de viata al consultatiei  
5   payment/       -- Stripe escrow si transfer  
6   video/         -- Daily.co video provider  
7   medic/         -- profil medici si clinici  
8   audit/         -- jurnal de audit  
9   webhook/       -- receptie Stripe/Daily webhooks  
10  config/        -- configuratii Spring (Security, Stripe,  
    CORS)  
11 src/main/resources/  
12   db/migration/  -- migrari Flyway (V1-V4)  
13   application.yml
```

5.2 Entități JPA

5.2.1 Entitatea Payment

Entitatea centrală a modului de plăți implementează o mașină de stare cu șase stări posibile. Blocarea optimistă prin `@Version` previne condițiile de concurență în operații critice (eliberare simultană din două thread-uri):

Listing 5.2: Entitatea Payment cu masina de stare

```
1 @Entity  
2 @Table(name = "payments")  
3 public class Payment {
```

```

4      @Id
5      private UUID id;
6
7      @Column(name = "booking_id", nullable = false)
8      private UUID bookingId;
9
10     // Suma in bani (1 RON = 100 bani) -- niciodata double
11     // sau float!
12     @Column(name = "amount_bani", nullable = false)
13     private long amountBani;
14
15     @Enumerated(EnumType.STRING)
16     @Column(nullable = false)
17     private PaymentState state;
18
19     @Column(name = "stripe_payment_intent_id")
20     private String stripePaymentIntentId;
21
22     @Column(name = "stripe_charge_id")
23     private String stripeChargeId;
24
25     @Column(name = "stripe_transfer_id")
26     private String stripeTransferId;
27
28     @Version
29     private long version; // blocare optimista -- previne
30     // race conditions
31 }
32
33 public enum PaymentState {
34     RESERVED, HELD, RELEASED, REFUNDED, DISPUTED, FAILED
35 }

```

5.2.2 Entitatea Consultation

Entitatea Consultation modelează sesiunea medicală completă, incluzând referințe la camera video, durata efectivă și motivul eșecului:

Listing 5.3: Entitatea Consultation

```

1  @Entity
2  @Table(name = "consultations")
3  public class Consultation {
4      @Id
5      private UUID id;
6
7      @OneToOne(fetch = FetchType.LAZY)
8      @JoinColumn(name = "booking_id")
9      private Booking booking;
10
11     @Enumerated(EnumType.STRING)

```

```

12     private ConsultationStatus status;
13
14     private String videoRoomId;
15     private String videoRoomUrl;
16
17     @Column(name = "started_at")
18     private Instant startedAt;
19
20     @Column(name = "ended_at")
21     private Instant endedAt;
22
23     @Column(name = "duration_seconds")
24     private Integer durationSeconds;
25
26     @Column(name = "failure_reason")
27     private String failureReason;
28
29     // Momentul la care fondurile se elibereaza automat
30     @Column(name = "release_at")
31     private Instant releaseAt;
32 }

```

5.3 Serviciile Aplicației

5.3.1 ConsultationServiceImpl

Serviciul de consultații gestionează ciclul de viață complet al sesiunii medicale, coordonând interacțiunile cu `PaymentService` și `VideoProvider`:

Listing 5.4: Metoda `start()` din `ConsultationService`

```

1  @Transactional
2  public ConsultationResponse start(UUID consultationId, UUID
   medicId) {
3      Consultation c = consultationRepository
4          .findById(consultationId)
5          .orElseThrow(() -> new NotFoundException("Not
   found"));
6
7      validateMedicOwnership(c, medicId);
8      validateState(c, ConsultationStatus.SCHEDULED);
9
10     // Create camera video prin VideoProvider (abstractizat)
11     VideoRoom room = videoProvider.createRoom(
12         consultationId,
13         Instant.now(),
14         c.getBooking().getScheduledEnd().plus(Duration.ofHours(2)),
15         RoomPrivacy.PRIVATE
16     );
17
18     // Emitere token medic (OWNER -- poate muta/ejecta)

```

```

19     VideoMeetingToken medicToken = videoProvider.issueToken(
20         consultationId, room.roomName(), medicId,
21         c.getBooking().getMedic().getDisplayName(),
22         ParticipantRole.OWNER, Duration.ofHours(3)
23     );
24
25     c.setVideoRoomId(room.roomName());
26     c.setVideoRoomUrl(room.roomUrl());
27
28     // Captura plata Stripe (escrow -> HELD)
29     stripePaymentService.capturePaymentIntent(
30         c.getBooking().getPayment().getStripePaymentIntentId()
31     );
32
33     auditLogService.log("CONSULTATION_STARTED", medicId,
34         consultationId, null, "IN_PROGRESS");
35
36     return ConsultationResponse.from(c, medicToken.token());
37 }

```

5.4 Gestionarea Idempotentă a Webhook-urilor

Un aspect critic al arhitecturii este gestionarea idempotentă a webhook-urilor. Orice webhook primit este mai întâi stocat în tabela `webhook_events` și procesat o singură dată, chiar dacă furnizorul retransmite evenimentul:

Listing 5.5: Pattern de idempotenta pentru webhooks

```

1 @PostMapping("/api/v1/webhooks/stripe")
2 public ResponseEntity<Void> handleStripeWebhook(
3     HttpServletRequest request,
4     @RequestHeader("Stripe-Signature") String signature)
5     {
6         // Citim body-ul RAW -- necesar pentru verificarea
7         semnaturii
8         byte[] rawBody = request.getInputStream().readAllBytes();
9         Event event = Webhook.constructEvent(rawBody, signature,
10             webhookSecret);
11
12         // Idempotenta: returnam 200 daca evenimentul a fost
13         deja procesat
14         if (webhookEventRepository.existsByIdAndStatus(
15             event.getId(), PROCESSED)) {
16             return ResponseEntity.ok().build();
17         }
18
19         // Persistam inainte de procesare
20         WebhookEvent record = webhookEventRepository.save(
21             WebhookEvent.pending(event.getId(), event.getType(),
22                 rawBody)

```



```

19     );
20
21     try {
22         dispatchStripeEvent(event);
23         record.markProcessed();
24     } catch (Exception e) {
25         record.markFailed(e.getMessage());
26         // Returnam 500 pentru ca Stripe sa reincearca
27         return ResponseEntity.internalServerError().build();
28     }
29
30     return ResponseEntity.ok().build();
31 }

```

5.5 Migrările Flyway

Schema bazei de date este gestionată prin patru migrări Flyway:

- **V1__baseline.sql:** schema de bază (users, bookings, consultations, payments, audit_logs);
- **V2__quartz_schema.sql:** tabelele necesare Quartz JDBC JobStore (PostgreSQL flavour);
- **V3__stripe_tables.sql:** medic_stripe_account, webhook_events, indecși Stripe;
- **V4__video_consultation.sql:** video_session_events, video_meeting_tokens, coloane noi pe consultations.

5.6 API REST

Toate endpoint-urile API sunt sub prefixul /api/v1, securizate cu JWT și cu control de acces bazat pe roluri prin @PreAuthorize. Erorile sunt returnate în formatul RFC 7807 application/problem+json cu un câmp traceId pentru corelarea logurilor. Tabelul 5.1 prezintă principalele endpoint-uri.

Metodă	Cale	Descriere
POST	/payments/intents	Creare PaymentIntent (rol PATIENT)
GET	/payments/{id}	Detalii plată (proprietar sau operator)
POST	/payments/{id}/refund	Rambursare (OPERATOR sau PATIENT în fereastră)
POST	/medics/me/stripe/onboarding	Onboarding Stripe (rol MEDIC)
GET	/medics/me/stripe/status	Status cont Stripe medic
GET	/medics/me/payouts	Istoric plăți medic (paginat)
GET	/consultations/{id}/join-token	Token video pentru participare
POST	/webhooks/stripe	Webhook Stripe (public, verificat HMAC)
POST	/webhooks/video	Webhook Daily.co (public, verificat HMAC)

Tabelul 5.1: Principalele endpoint-uri API REST

Capitolul 6

Sistemul de Plăți Stripe Connect

6.1 Modelul de Plată Escrow

MediConnect implementează un model de plată escrow în care fondurile pacientului sunt capturate imediat la rezervare și transferate medicului doar după finalizarea cu succes a consultației [Str24]. Acest model protejează ambele părți și este conform cu cerințele PSD2 [Par15].

Mașina de stare a plății parcurge următoarele tranziții:

RESERVED → HELD → RELEASED

RESERVED → FAILED

HELD → REFUNDED

HELD → DISPUTED

6.2 Decizii de Arhitectură Stripe

Următoarele decizii tehnice au fost înregistrate ca decizii definitive ale arhitecturii și sunt prezentate în Tabelul 6.1.

Decizie	Valoare
Model cont Stripe	Connect Express pentru medici
Model de încărcare	PaymentIntent pe contul platformei → Transfer către medic
Valută	RON (unitate minimă: bani, 1 RON = 100 bani)
Mod captură	automatic (charge-now-transfer-later)
3DS / SCA	automatic_payment_methods.enabled = true
Tole-ranță semnătură webhook	300 secunde (default Stripe)
Fereastră dispute	120 minute după finalul consultației
Fereastră anulare	2 ore înainte de start = rambursare totală

Tabelul 6.1: Decizii de arhitectură Stripe

6.3 Chei de Idempotență

Toate operațiunile Stripe folosesc chei de idempotență pentru a preveni dubbele procesări în cazul retrimitărilor de rețea:

- Creare PaymentIntent: `pi:<bookingId>`
- Transfer către medic: `<paymentId>:release`
- Rambursare: `<paymentId>:refund:<reason>`
- Inversare transfer: `<paymentId>:reverse`

6.4 Matrice de Scenarii de Eșec

Politicile de gestionare a eșecurilor sunt documentate exhaustiv și implementate în serviciile de business, așa cum este prezentat în Tabelul 6.2.

Scenariu	Pacient	Medic	Platformă
Anulare cu 2h+ înainte	Rambursare totală	Nimic	Fără taxă
Anulare < 2h înainte	Fără rambursare	Plată totală (minus comision)	Taxă percepută
Pacient absent (no-show)	Fără rambursare	Plată totală	Taxă percepută
Medic absent (no-show)	Rambursare totală + credit	Nimic + penalizare	Fără taxă
Eșec tehnic (platformă)	Rambursare totală	Compensatie SLA	Platformă absoarbe

Tabelul 6.2: Matrice scenarii de eșec — politici financiare

6.5 Onboarding Medic pe Stripe Connect

Medicii trebuie să parcurgă procesul de onboarding Stripe Express înainte de a putea primi plăți:

1. Medicul accesează endpoint-ul `POST /medics/me/stripe/onboarding`;
2. Backend-ul creează un cont Stripe Express cu `country='RO'`, `default_currency='ron'`;
3. Stripe returnează un URL de onboarding la care medicul este redirijat;
4. La revenire, backend-ul sincronizează starea contului (`chargesEnabled`, `payoutsEnabled`);
5. Webhook-ul `account.updated` actualizează în timp real flag-urile din tabela `medic_stripe_account`.

6.6 Evenimentele Webhook Stripe Implementate

Tabelul 6.3 prezintă toate evenimentele Stripe implementate cu acțiunile corespunzătoare.

Eveniment Stripe	Acțiune Backend
payment_intent.succeeded	RESERVED → HELD, persist chargeId
payment_intent.payment_failed	RESERVED → FAILED, eliberare slot
payment_intent.canceled	RESERVED → FAILED
charge.refunded	Înregistrare refundId, tranziție la REFUNDED
charge.dispute.created	→ DISPUTED, oprire eliberare Quartz
charge.dispute.closed	Reconciliere conform outcome
transfer.created	Înregistrare transferId pe payment
account.updated	Re-sincronizare MedicStripeAccount
account.application.deauthorized	Marcare medic offboarded
payout.paid / payout.failed	Audit log pentru istoricul plăților medic

Tabelul 6.3: Evenimente webhook Stripe implementate

Capitolul 7

Modulul de Consultații Video

7.1 Arhitectura VideoProvider

Principiul central al modulului video este abstractizarea furnizorului prin interfata `VideoProvider`, care permite schimbarea furnizorului WebRTC fără modificarea logicii de business [Dai24]:

Listing 7.1: Interfata `VideoProvider`

```
1 public interface VideoProvider {
2     /**
3      * Creeaza o camera video pentru o consultatie.
4      * Idempotent pe consultationId: al doilea apel
5      * returneaza camera existenta.
6      */
7     VideoRoom createRoom(UUID consultationId,
8                           Instant earliestJoinAt,
9                           Instant expiresAt,
10                          RoomPrivacy privacy);
11
12     /**
13      * Emite un token de acces cu durata limitata.
14      * OWNER pentru medic (poate muta/ejecta/inregistra).
15      * PARTICIPANT pentru pacient.
16      */
17     VideoMeetingToken issueToken(UUID consultationId,
18                                   String roomName,
19                                   UUID userId,
20                                   String displayName,
21                                   ParticipantRole role,
22                                   Duration ttl);
23
24     void deleteRoom(String roomName);
25 }
```

Implementarea concretă `DailyVideoProvider` utilizează `WebClient` (reactive HTTP client) pentru apeluri către API-ul Daily.co. Un element cheie este pinarea regiunii la `eu-west` (Frankfurt/Dublin) pentru conformitate GDPR — datele audio/video ale consultației rămân în Uniunea Europeană.

7.2 Evenimentele Webhook Video

Evenimentele video de la Daily.co urmăresc același pattern de idempotență ca și webhook-urile Stripe. Tabelul 7.1 prezintă evenimentele implementate.

Eveniment Daily	Acțiune Backend
meeting.started	Consultation.status = IN_PROGRESS, startedAt = timestamp
participant.joined	Audit log, SSE push (Faza 3)
participant.left	Planificare job debounce (90s) pentru completare
meeting.ended	ConsultationService.complete(durationSeconds)
recording.ready	(Faza 3) Copiere în S3, scriere recording_url

Tabelul 7.1: Evenimente Daily.co și acțiunile corespunzătoare

7.3 Automatizarea No-Show

ConsultationNoShowJob este planificat de Quartz să ruleze la 10 minute după ora programată a fiecărei consultații:

- Dacă starea este SCHEDULED și medicul nu s-a alăturat → MEDIC_NO_SHOW: rambursare totală + penalizare medic;
- Dacă starea este IN_PROGRESS dar un singur participant s-a conectat → PATIENT_NO_SHOW: plata integrală către medic;
- Eșec tehnic (durată < 5 minute) → TECHNICAL_FAILURE: rambursare totală.

7.4 Securitatea Tokenurilor Video

Tokenurile de acces video sunt emise cu următoarele proprietăți de securitate: durată de viață de 15 minute, JTI (JWT ID) unic stocat pentru posibilitate de revocare, tokenurile nu sunt persistate în localStorage, ci stocate în memoria componentei React. La revindicarea paginii în timpul unui apel, un nou token este emis automat.

7.5 Schema Bazei de Date pentru Video

Modulul video introduce două tabele noi prin migrația Flyway V4:

Listing 7.2: Schema pentru modulul video

```
1 -- Istoric append-only al evenimentelor video
2 CREATE TABLE video_session_events (
3     id                UUID PRIMARY KEY DEFAULT
        gen_random_uuid(),
```

```

4      consultation_id UUID NOT NULL REFERENCES
        consultations(id),
5      event_type      VARCHAR(32) NOT NULL,
6      actor_user_id   UUID,
7      payload         JSONB NOT NULL,
8      occurred_at     TIMESTAMPTZ NOT NULL,
9      received_at     TIMESTAMPTZ NOT NULL DEFAULT now()
10 );
11 CREATE INDEX ix_vse_consultation
12     ON video_session_events(consultation_id, occurred_at);
13
14 -- Tokenuri emise pentru audit si revocare
15 CREATE TABLE video_meeting_tokens (
16     id                UUID PRIMARY KEY DEFAULT
        gen_random_uuid(),
17     consultation_id   UUID NOT NULL REFERENCES
        consultations(id),
18     user_id          UUID NOT NULL REFERENCES users(id),
19     role              VARCHAR(16) NOT NULL,
20     token_jti        VARCHAR(64) NOT NULL UNIQUE,
21     issued_at        TIMESTAMPTZ NOT NULL,
22     expires_at       TIMESTAMPTZ NOT NULL,
23     revoked_at       TIMESTAMPTZ
24 );

```


Capitolul 8

Implementarea Frontend-ului React/TypeScript

8.1 Arhitectura Frontend

Frontend-ul MediConnect este o aplicație SPA (Single Page Application) construită cu React 19 și TypeScript în mod strict [BP20, Mic24]. Rutarea este implementată printr-un router custom în `App.tsx`, bazat pe un union tip discriminant, fără dependențe externe de rutare.

8.2 Integrarea Stripe Elements

Fluxul de plată este implementat prin Stripe Elements, o soluție de UI securizată care izolează datele de card într-un iframe securizat gestionat de Stripe:

Listing 8.1: StripeProvider.tsx

```
1 import { Elements } from '@stripe/react-stripe-js';
2 import { loadStripe } from '@stripe/stripe-js';
3
4 // loadStripe este chemat o singura data la nivel de modul
5 // (memoizat)
6
7 const stripePromise =
8   loadStripe(env.VITE_STRIPE_PUBLISHABLE_KEY);
9
10 export function StripeProvider({ clientSecret, children }) {
11   const { i18n } = useTranslation();
12   return (
13     <Elements
14       stripe={stripePromise}
15       options={{
16         clientSecret,
17         locale: i18n.language as 'ro' | 'en',
18         appearance: {
19           theme: 'stripe',
20           variables: {
21             colorPrimary: '#0F766E', // deep teal -- brand
22             token
```

```

19         colorDanger: '#EF4444',    // error red
20         fontFamily: 'system-ui, sans-serif',
21     },
22 },
23 }}
24 >
25     {children}
26 </Elements>
27 );
28 }

```

8.3 Componenta StripeStatusBadge

`StripeStatusBadge` este o componentă unică care afișează starea plății cu culori și texte diferite în funcție de rolul utilizatorului (Tabelul 8.1).

Stare	Culoare	Pacient	Medic	Operator
RESERVED	amber	Așteptă confirmare	În așteptare	vizibil
HELD	teal	Plătit — în escrow	Câștig în așteptare	vizibil
RELEASED	green	Finalizat	Plătit	vizibil
REFUNDED	slate	Rambursat	Rambursat	vizibil
DISPUTED	red	ascuns	În revizuire	vizibil
FAILED	red	Plată eșuată	ascuns	vizibil

Tabelul 8.1: Stare — culoare — text per rol (`StripeStatusBadge`)

8.4 Workspace-ul de Consultație al Medicului

`MedicConsultationWorkspace.tsx` este componenta principală a interfeței de consultație video. Aceasta înlocuiește `iframe`-ul static cu o integrare nativă Daily React SDK:

Listing 8.2: Fragment `MedicConsultationWorkspace`

```

1 import {
2     DailyProvider, useParticipantIds,
3     useDailyEvent, useLocalParticipant
4 } from '@daily-co/daily-react';
5
6 export function MedicConsultationWorkspace({ consultationId
7     }) {
8     const { data: joinData } = useJoinToken(consultationId,
9         'OWNER');
10    const { mutate: endConsultation } = useEndConsultation();
11
12    if (!joinData) return <LoadingScreen />;
13
14    return (
15        <DailyProvider

```

```

14     url={joinData.roomUrl}
15     token={joinData.token}
16     subscribeToTracksAutomatically={true}
17   >
18     <div className="flex h-screen">
19       {/* Panoul video -- stanga */}
20       <div className="flex-1">
21         <VideoGrid />
22         <VideoConsoleStrip
23           onEnd={() => endConsultation(consultationId)}
24         />
25       </div>
26       {/* Panoul notes/prescriptii -- dreapta */}
27       <aside className="w-[340px] border-1">
28         <NotesPrescriptionsPanel
29           consultationId={consultationId} />
30       </aside>
31     </div>
32   </DailyProvider>
33 );
34 }

```

8.5 Pagina de Consultație a Pacientului

`PatientConsultationPage.tsx` oferă pacientului o interfață simplificată de consultație video, cu un lobby pre-apel și auto-test de microfon/cameră. Funcționalitățile cheie includ: lobby pre-apel cu testul de dispozitive Daily, numărătoare inversă până la începerea consultației, butonul Join activ cu 5 minute înainte de ora programată și panou read-only cu prescripțiile și sumarul consultației.

8.6 Controlul Accesului cu CASL

Permișiunile UI sunt gestionate cu CASL [Sto24], o bibliotecă de autorizare izomorfă care reflectă politicile de acces din backend:

Listing 8.3: Definire abilitati CASL

```

1 export function defineAbilityFor(user) {
2   const { can, build } = new
      AbilityBuilder(createMongoAbility);
3
4   if (user.roles.includes('PATIENT')) {
5     can('read', 'OwnBookings');
6     can('create', 'Booking');
7     can('cancel', 'OwnBooking');
8   }
9
10  if (user.roles.includes('MEDIC')) {
11    can('manage', 'OwnConsultations');

```

```
12     can('manage', 'OwnPayouts');
13     can('write', 'ConsultationNotes');
14     can('create', 'Prescription');
15 }
16
17 if (user.roles.includes('OPERATOR')) {
18     can('manage', 'all');
19 }
20
21 return build();
22 }
```

8.7 Internaționalizarea Interfeței

Toate string-urile vizibile utilizatorului sunt externalizate în fișiere de traducere prin `i18next`, cu suport complet pentru română și engleză. Toate string-urile legate de Stripe (starea plăților, mesajele de eroare) sunt cheiate în namespace-ul `payments`, `payouts` și `stripe`, fără niciun string hardcodat în componente.

Capitolul 9

Securitate și Conformitate GDPR

9.1 Strategia de Securitate

MediConnect implementează o strategie de securitate în adâncime (defense in depth) cu multiple straturi de protecție:

1. **Autentificare:** JWT cu HS256, validare la fiecare request;
2. **Autorizare:** RBAC prin `@PreAuthorize` pe endpoint-uri și CASL în UI;
3. **Criptare date la repaus:** AES-256-GCM pentru note medicale și prescripții;
4. **Criptare în tranzit:** TLS 1.3 obligatoriu pentru toate conexiunile;
5. **Validare input:** Bean Validation (Jakarta) pe toate DTO-urile de intrare;
6. **CORS:** restricționat la originile configurate prin variabile de mediu.

9.2 Criptarea Datelor Medicale

Notele medicale și prescripțiile sunt criptate înainte de stocare folosind AES-256-GCM, un algoritm de criptare simetrică autenticată, care oferă atât confidențialitate cât și integritate:

- Cheile de criptare sunt citite exclusiv din variabile de mediu (`APP_ENCRYPTION_KEY_PRIMARY`, `APP_ENCRYPTION_KEY_SECONDARY`);
- Rotația cheilor este suportată prin sistemul dual de chei primară/secundară;
- Aplicația eșuează la pornire (fail-fast) dacă cheile lipsesc sau au lungimea incorectă (32 bytes Base64).

9.3 Conformitate GDPR

MediConnect procesează date cu caracter special conform Art. 9 GDPR (date de sănătate) [Par16]. Măsurile tehnice și organizaționale implementate includ:

- **Minimizarea datelor:** colectarea strictă a datelor necesare consultației;

- **Rezidența EU a datelor video:** sesiunile Daily.co sunt pin-ate la PoP-urile Frankfurt/Dublin;
- **Dreptul la ștergere:** API-uri dedicate pentru ștergerea datelor la cererea pacientului;
- **Auditabilitate:** jurnal complet imutabil al tuturor accesurilor la date medicale;
- **Pseudonimizare:** identificatorii externi nu conțin date personale;
- **Redactare log-uri:** filtre automate care previn logarea datelor sensibile.

9.4 Conformitate PSD2/SCA

Directiva PSD2 impune autentificarea puternică a clientului (SCA) pentru plățile online [Par15]. MediConnect asigură conformitatea prin:

- `automatic_payment_methods.enabled = true` — Stripe detectează automat necesitatea 3DS;
- Suport complet pentru redirect-uri 3DS în fluxul de plată;
- Pagina de rezultat `/bookings/{id}/result` gestionează toate stările post-redirect.

9.5 Securitatea Webhook-urilor

Endpoint-urile webhook au măsuri speciale de securitate: verificarea semnăturii HMAC-SHA256 pe body-ul RAW (nu pe obiectul deserializat), cu fereastră de toleranță de 300 secunde pentru Stripe și 5 minute pentru Daily. Endpoint-urile webhook sunt publice (fără JWT) dar excluse din lista CORS — Stripe și Daily apelează server-to-server. Orice request cu semnătură invalidă returnează 400 fără informații despre motivul respingerii.

Capitolul 10

Testarea și Validarea Sistemului

10.1 Strategia de Testare

MediConnect adoptă o strategie de testare pe mai multe niveluri, conform principiilor Testing Pyramid:

- **Teste unitare:** logica de business, mașina de stare a plăților, formatarea valorilor monetare;
- **Teste de integrare:** endpoint-uri REST cu Testcontainers (PostgreSQL real), handler-e webhook cu payload-uri reale Stripe;
- **Teste end-to-end:** fluxul complet de rezervare + plată + consultație video contra backend-ului în modul test Stripe.

10.2 Testele Unitare Backend

Testele unitare verifică toate tranzițiile permise și interzise ale mașinii de stare a plăților, incluzând scenariile de concurență:

Listing 10.1: Test concurența pentru eliberare plată

```
1 @Test
2 void concurrentRelease_onlyOneSucceeds() throws
   InterruptedException {
3     Payment payment = PaymentTestFactory.held();
4     CountdownLatch latch = new CountdownLatch(1);
5     AtomicInteger successCount = new AtomicInteger(0);
6     AtomicInteger conflictCount = new AtomicInteger(0);
7
8     // Doua thread-uri incearca eliberarea simultan
9     List<Thread> threads = IntStream.range(0, 2)
10        .mapToObj(i -> Thread.ofVirtual().start(() -> {
11            try {
12                latch.await();
13                paymentService.release(payment.getId());
14                successCount.incrementAndGet();
15            } catch (OptimisticLockingFailureException e) {
```

```

16         conflictCount.incrementAndGet();
17     } catch (InterruptedException ex) {
18         Thread.currentThread().interrupt();
19     }
20     })).toList();
21
22     latch.countDown();
23     threads.forEach(t -> {
24         try { t.join(5000); } catch (InterruptedException e)
25             {}
26     });
27
28     // Exact o eliberare reusita, un conflict
29     assertThat(successCount.get()).isEqualTo(1);
30     assertThat(conflictCount.get()).isEqualTo(1);
31 }

```

10.3 Testele de Integrare

Testele de integrare folosesc Testcontainers pentru a porni un PostgreSQL real în Docker și MockWebServer pentru a simula răspunsurile API Stripe. Toate cele 11 tipuri de webhook Stripe sunt acoperite cu payload-uri reale. Cele 5 scenarii din matricea de eșec sunt testate end-to-end cu verificarea fluxului de bani. Migrările Flyway se aplică împotriva unei instanțe PostgreSQL proaspete la fiecare rulare de test.

10.4 Testele Frontend

Frontend-ul utilizează următoarea stivă de testare:

- **Vitest + React Testing Library:** teste pentru componente UI și hook-uri custom;
- **MSW (Mock Service Worker):** interceptarea apelurilor API în teste, fără modificarea codului de producție;
- **Testul de polling:** verifică că pagina de rezultat poll-ează corect backend-ul dacă starea este încă RESERVED.

10.5 Carduri de Test Stripe

Următoarele carduri de test sunt utilizate pentru validarea fluxurilor de plată în mediul Stripe test mode (Tabelul 10.1).

10.6 Criterii de Acceptanță

Proiectul este considerat finalizat când toate criteriile următoare sunt îndeplinite:

Număr card	Comportament
4242 4242 4242 4242	Plată reușită
4000 0027 6000 3184	Necesită autentificare 3DS
4000 0000 0000 0002	Card respins
4000 0000 0000 9995	Fonduri insuficiente

Tabelul 10.1: Carduri de test Stripe

1. Ambele ramuri se compilează și trec suita completă de teste în CI;
2. Backend-ul compilează cu `mvn -B verify` și expune toate endpoint-urile din API;
3. Webhook handler-ul verifică semnătura pe body-ul RAW și este idempotent pe event id;
4. Frontend-ul se construiește cu `npm run build` fără erori TypeScript;
5. Quartz folosește JDBC JobStore în mod clustered; job-ul de auto-release se reprogramează după restart;
6. Niciun secret nu este commitat; toate cheile vin din variabile de mediu;
7. Toate tranzițiile de stare a plății sunt persistate cu `@Version`; tentativele de eliberare concurentă rezultă în exact un succes;
8. Interfata este în română implicit și cade back pe engleză; schimbarea limbii actualizează toate badge-urile;
9. Cele 5 scenarii de eșec au fiecare câte un test de integrare.

Capitolul 11

Ceea ce a Fost Realizat până în Prezent

11.1 Stadiul Actual al Implementării

La momentul redactării acestei lucrări, platforma MediConnect a atins următoarele jaloane de implementare. Componenta de backend este completă, incluzând: infrastructura de bază Spring Boot 4.0, Java 21, PostgreSQL, Flyway; sistemul de autentificare JWT cu Spring Security și toate cele trei roluri; modulul de rezervări cu gestionarea sloturilor orare; ciclul de viață al Consultației cu mașina de stare completă; toate entitățile JPA cu indecșii necesari; cele 4 migrări Flyway; Quartz configurat cu JDBC JobStore cluster-safe.

Implementarea Stripe Connect Express este completă, cuprinzând: onboarding medici, `PaymentService.reserve()` cu idempotență, `PaymentService.release()` cu ferestră de 120 minute, `PaymentService.refund()` cu Transfer Reversal, `PaymentService.handleDispute()`, webhook Stripe cu toate cele 11 tipuri de evenimente și `AutoReleaseConsultationJob` Quartz.

Implementarea video Daily.co este completă: `DailyVideoProvider` cu `WebClient`, `VideoWebhookController` cu toate evenimentele, `ConsultationNoShowJob` Quartz, stocare `video_session_events` și `video_meeting_tokens`, `AuditLogService` pentru toate evenimentele, criptare AES-256-GCM pentru note medicale, API REST complet cu formatul RFC 7807 și suita completă de teste (unit + integrare).

Componenta de frontend este completă, incluzând: setup Vite + React 19 + TypeScript + Tailwind v4 + shadcn/ui; autentificare JWT cu Zustand store; `BookingSheet` cu selecție medic/slot/serviciu; integrare Stripe Elements completă cu `StripeProvider`, pas de plată, pagină de rezultat, `StripeStatusBadge`, pagini onboarding medic și payouts; integrare Daily.co completă cu `MedicConsultationWorkspace`, `PatientConsultationPage`, `VideoConsoleStrip` wire-uit la daily-react hooks, `ConsultationFailureBanner`; internaționalizare completă ro/en cu i18next; autorizare CASL per rol; teste Vitest + MSW handlers.

11.2 Organizarea Ramurilor de Lucru

Toată implementarea a fost realizată pe ramuri dedicate, separat de ramura principală:

- Backend: `feature/improvements-2-stripe` (Stripe Connect + Daily.co video);
- Frontend: `feature/improvements-4-stripe` (Stripe Elements + Daily React SDK);
- Pull request-uri draft create pentru ambele ramuri, cu checklist complet al fazelor A-G.

Capitolul 12

Concluzii și Direcții Viitoare de Dezvoltare

12.1 Concluzii

Această lucrare de disertație a prezentat MediConnect, o platformă comprehensivă de telemedicină care abordează nevoile sistemului de sănătate digital din România printr-o abordare tehnică modernă și riguroasă.

Principalele realizări ale proiectului sunt:

1. **Arhitectură robustă și extensibilă:** proiectarea unui monolit modular care separă clar responsabilitățile domeniilor funcționale, permițând evoluția către microservicii fără refactorizare majoră;
2. **Model de plată escrow inovator:** primul sistem de plată escrow pentru consultații medicale pe piața românească, care protejează simultan pacientul (rambursare garantată în caz de eșec) și medicul (plată garantată pentru servicii prestate);
3. **Conformitate legală completă:** GDPR (Art. 9 date speciale de sănătate), PSD2/SCA și norme medicale românești sunt respectate prin măsuri tehnice documentate;
4. **Infrastructură video GDPR-compliantă:** integrarea Daily.co cu rezidența EU a datelor video, criptare HIPAA-grade implicită și SDK React modern;
5. **Automatizare robustă:** sistemul de job-uri Quartz JDBC gestionează automat eliberarea fondurilor și scenariile no-show, fără intervenție umană;
6. **Experiență utilizator localizată:** interfața complet internaționalizată (ro/en) cu componente accesibile și design responsive.

MediConnect demonstrează că este posibilă construirea unei platforme de telemedicină mature, sigure și conforme legal într-un cadru de dezvoltare modern bazat pe tehnologii open-source și servicii cloud de înaltă disponibilitate. Arhitectura aleasă echilibrează pragmatismul MVP-ului cu viziunea pe termen lung a unui sistem scalabil, oferind o bază solidă pentru evoluția platformei.

12.2 Direcții Viitoare de Dezvoltare

12.2.1 Faza 2 — Facturare și Taxă (Planificat)

- Integrarea Stripe Invoices pentru emiterea automată a facturilor fiscale conforme;
- Stripe Tax pentru calculul automat al TVA românesc pe servicii medicale;
- Export date contabile către sisteme ERP (Saga, WinMentor);
- API de raportare către CNAS pentru rambursarea consultațiilor telemedicale.

12.2.2 Faza 3 — Consultații Instant și Semnalizare Real-Time

- Consultații instant (on-demand) fără programare prealabilă;
- Sistem de routing Layer 2 pentru conectarea rapidă cu medicii disponibili;
- SSE (Server-Sent Events) pentru notificări în timp real;
- Înregistrarea audio/video a consultațiilor (cu consimțământul explicit al ambelor părți);
- Migrație la TanStack Router pentru rutare tipată.

12.2.3 Faza 4 — Inteligență Artificială Avansată

Inteligența artificială reprezintă cea mai transformătoare direcție de evoluție a platformei [Top19]:

- **Asistent AI de documentație:** utilizarea modelelor LLM (Claude API, GPT-4) pentru transcrierea automată a consultației, generarea de sugestii de diagnostic, completarea automată a fișei de consultație și detectarea interacțiunilor medicamentoase în prescripții;
- **Analiză predictivă:** algoritmi ML pentru prezicerea no-show-urilor și optimizarea programului medicilor;
- **Triaaj AI:** sistem inteligent de triaj inițial care îndrumă pacientul către specialitatea potrivită pe baza simptomelor descrise;
- **Detectie fraud:** identificarea tranzacțiilor suspecte prin algoritmi de anomalie specific domeniului medical.

12.2.4 Faza 5 — Scalabilitate și Enterprise

- Migrare la arhitectură microservicii cu Kubernetes (EKS/GKE);
- Multi-tenancy pentru rețele de clinici și spitale;
- Aplicații mobile native (React Native / Flutter);
- Integrare cu dosarul electronic de sănătate (DES) al pacientului;
- API-uri pentru integrarea cu sisteme HIS (Hospital Information System) existente;
- Conformitate SOC 2 Type II și ISO 27001 pentru piața enterprise.

12.2.5 Direcții de Cercetare

- Implementarea unui model de machine learning pentru detectarea anomaliilor specifice domeniului medical în fluxurile de plăți;
- Cercetare în domeniul explainability AI (XAI) pentru sistemele de suport decizie medicală;
- Studiu privind impactul telemedicinii asupra accesibilității serviciilor medicale în mediul rural românesc.

Bibliography

- [BP20] Alex Banks and Eve Porcello. *Learning React: Modern Patterns for Developing React Apps*. O'Reilly Media, 2020.
- [Dai24] Daily.co. Daily.co video api documentation. <https://docs.daily.co>, 2024. Accesat: mai 2025.
- [Eva03] Eric Evans. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley Professional, 2003.
- [JBS15] M. Jones, J. Bradley, and N. Sakimura. Json web token (jwt) – rfc 7519. <https://tools.ietf.org/html/rfc7519>, 2015. IETF RFC 7519.
- [Kat24] Daishi Kato. Zustand – bear necessities for state management in react. <https://zustand-demo.pmnd.rs/>, 2024. Accesat: mai 2025.
- [Lin24] Tanner Linsley. Tanstack query – powerful asynchronous state management. <https://tanstack.com/query/latest>, 2024. Accesat: mai 2025.
- [Mar17] Robert C. Martin. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017.
- [Mic24] Microsoft Corporation. Typescript documentation. <https://www.typescriptlang.org/docs/>, 2024. Accesat: mai 2025.
- [Par15] Parlamentul European și Consiliul Uniunii Europene. Directiva (ue) 2015/2366 privind serviciile de plată în cadrul pieței interne (psd2). *Jurnalul Oficial al Uniunii Europene*, L 337:35–127, 2015.
- [Par16] Parlamentul European și Consiliul Uniunii Europene. Regulamentul (ue) 2016/679 privind protecția persoanelor fizice în ceea ce privește prelucrarea datelor cu caracter personal. *Jurnalul Oficial al Uniunii Europene*, L 119:1–88, 2016.
- [Red24] Redgate Software. Flyway documentation – database migrations made easy. <https://flywaydb.org/documentation/>, 2024. Accesat: mai 2025.
- [Ric18] Chris Richardson. *Microservices Patterns: With Examples in Java*. Manning Publications, 2018.
- [SSP15] Branislav Sredojev, Dragan Samardzija, and Predrag Poscic. Webrtc technology overview and signaling solution design and implementation. *38th International Convention on Information and Communication Technology, Electronics and Microelectronics*, pages 1056–1061, 2015.

- [Sto24] Sergii Stotskyi. Casl – isomorphic authorization javascript library. <https://casl.js.org/>, 2024. Accesat: mai 2025.
- [Str24] Stripe Inc. Stripe connect documentation. <https://stripe.com/docs/connect>, 2024. Accesat: mai 2025.
- [Ter24] Terracotta Inc. Quartz scheduler documentation. <http://www.quartz-scheduler.org/documentation/>, 2024. Accesat: mai 2025.
- [The24] The PostgreSQL Global Development Group. Postgresql 16 documentation. <https://www.postgresql.org/docs/16/>, 2024. Accesat: mai 2025.
- [Top19] Eric J. Topol. High-performance medicine: the convergence of human and artificial intelligence. In *Nature Medicine*, volume 25, pages 44–56, 2019.
- [VMw24] VMware Inc. Spring security reference documentation. <https://docs.spring.io/spring-security/reference/>, 2024. Accesat: mai 2025.
- [Wal16] Craig Walls. *Spring Boot in Action*. Manning Publications, 2016.
- [WFC⁺20] Jedrek Wosik, Marat Fudim, Blake Cameron, Ziad F. Gellad, Alex Cho, Donna Phinney, Shannon Curtis, Matthew Roman, Eric G. Poon, Jeffrey Ferranti, Jason N. Katz, and James Tchong. Telehealth transformation: Covid-19 and the rise of virtual care. *Journal of the American Medical Informatics Association*, 27(6):957–962, 2020.
- [WR24] Adam Wathan and Jonathan Reinink. Tailwind css – a utility-first css framework. <https://tailwindcss.com/docs>, 2024. Accesat: mai 2025.
- [You24] Evan You. Vite – next generation frontend tooling. <https://vitejs.dev/>, 2024. Accesat: mai 2025.

Anexa A

Variabile de Mediu Necesare

Variabilă	Descriere	Oblig.
STRIPE_API_KEY	Cheia API Stripe (secret)	Da
STRIPE_WEBHOOK_SECRET	Secret verificare webhook Stripe	Da
DAILY_API_KEY	Cheia API Daily.co	Da
DAILY_WEBHOOK_SECRET	Secret verificare webhook Daily	Da
APP_ENCRYPTION_KEY_PRIMARY	Cheie AES-256-GCM primară (32 bytes Base64)	Da
APP_ENCRYPTION_KEY_SECONDARY	Cheie AES-256-GCM secundară (rotație)	Da
JWT_SECRET	Secretul pentru semnarea JWT	Da
DB_URL	URL PostgreSQL	Da
DB_PASSWORD	Parolă PostgreSQL	Da
APP_CORS_ALLOWED_ORIGINS	Origini permise CORS	Da
VITE_STRIPE_PUBLISHABLE_KEY	Cheia publică Stripe (frontend)	Da
VITE_FEATURE_STRIPE	Flag feature Stripe (boolean)	Nu

Tabelul A.1: Variabile de mediu necesare MediConnect

Anexa B

Utilizarea Instrumentelor de Inteligență Artificială Generativă

În această anexă este prezentată o sinteză a modului în care au fost utilizate instrumentele de inteligență artificială generativă în cadrul lucrării.

Instrument AI	Scopul utilizării	Secțiuni	Observații
Claude (Anthropic)	Generare cod boilerplate Spring Boot	Cap. 5, 6, 7	Verificat manual
	Asistare redactare	Toată lucrarea	Revizuit de autor
GitHub Copilot	Completare cod TypeScript	Cap. 8	Verificat prin teste
ChatGPT	Cercetare bibliografie inițială	Cap. 2	Surse verificate manual

Tabelul B.1: Sinteza utilizării instrumentelor AI generative

B.1 Detalii privind Utilizarea Claude (Anthropic)

Claude a fost utilizat în două moduri principale: (1) Generare cod — prompt-uri specifice pentru generarea structurii inițiale a serviciilor Spring Boot. Tot codul generat a fost revizuit, testat și adaptat; (2) Asistare documentație — structurarea secțiunilor de arhitectură. Conținutul a fost verificat pentru acuratețe tehnică și adaptat stilului lucrării.

B.2 Detalii privind Utilizarea GitHub Copilot

GitHub Copilot a fost utilizat în editorul de cod pentru sugestii de completare automată în special pentru codul TypeScript/React. Toate sugestiile au fost evaluate critic și integrate numai după verificarea corectitudinii.

Anexa C

Ghid de Rulare Locală

C.1 Pornire Backend

Listing C.1: Comenzi pornire backend

```
1 cd healthCare
2 git checkout feature/improvements-2-stripe
3 export STRIPE_API_KEY=sk_test_...
4 export DAILY_API_KEY=...
5 export JWT_SECRET=your-secret-here
6 docker compose up -d postgres
7 ./mvnw spring-boot:run
8 # In alt terminal -- forward webhook Stripe
9 stripe listen --forward-to
   localhost:8080/api/v1/webhooks/stripe
```

C.2 Pornire Frontend

Listing C.2: Comenzi pornire frontend

```
1 cd my-app
2 git checkout feature/improvements-4-stripe
3 cp .env.example .env.local
4 # Editati VITE_STRIPE_PUBLISHABLE_KEY=pk_test_...
5 npm install
6 npm run dev
7 # Rulare teste
8 npm run lint && npm run typecheck && npm test
```